CS211 Group Project: 2048 and Luhn Validation

Project overview

Complete both parts of this project in a group of 2-3 students: (A) a playable 2048 console game, and (B) a Luhn checksum validation program. Submit both programs together in one documented source-code package. Both parts are required; they are separate programs and do not need to share a menu.

Project duration: three weeks from release. The exact submission date and time will be posted on Blackboard. The practice-problems lab has its own deadline and GTA check-off requirements.

Use standard C++17, functions, loops, conditions, arrays, and text-file input/output. Use a 5x5 array for 2048. You may use a string or character array for the Luhn input. A graphical interface and third-party libraries are not required.

Groups and responsibilities

- Register all members in the course Google Sheet under the correct MW or TTh section.
- Form groups only within your own session. MW and TTh students may not form mixed-session groups.
- Submit one ZIP per group. Put names/WSUIDs in group_info.txt for Blackboard only; use names and GitHub usernames in public files.
- Every member must contribute equally, review the integrated code, and understand both programs. Members may lead different tasks, but must review and test work beyond their own part.
- The project receives a group grade. A student who did not contribute or help with the work will not receive the group grade, as stated in the course group-work instructions.
- Be prepared to demonstrate both programs and explain code, dry runs, and your own contribution. Any project demonstration schedule will be announced separately.

Three-week work plan

Stage	Target
Week 1	Set up Git/GitHub and the group repository. Merge setup and left-movement PRs; begin the remaining 2048 directions. Share the repository link through Blackboard.
Week 2	Merge the remaining 2048 features and save/load after review; complete and review Luhn. Test the integrated code and share a short progress update.
Week 3	Complete final tests and documentation. Review and merge the packaging PR, update main, and test the Blackboard ZIP before submission.

Read the Part A and Part B descriptions first. Then follow the Git/GitHub setup and six project checkpoints later in this handout. Share weekly progress through Blackboard; submit one final group package by the deadline.

Grading: 100 points

Area	Points
2048: correct sliding and merging in four directions	30
2048: game loop, tile insertion, score, controls, win/game-over	15
2048: save/load, validation, and preservation after failed loads	15
Luhn: input validation, checksum, repeatable interaction	20
Organization, buildable package, README, contributions	10
Test evidence, dry runs, and Luhn diagrams	10
Total	100

Part A: 2048 - movement and game control

Starting point

Use the supplied main.cpp and 2048_helper.h as a starting point. The movement functions are unfinished. You must also correct the main loop, random insertion behavior, and file organization. This handout governs the requirements where older handouts or starter behavior differ.

The board is a 5x5 integer array. Each cell contains zero for empty, or a power of two starting at 2. A new game clears the board, resets score and successful-move count to zero, and places two tiles of value 2 in different randomly chosen empty cells.

Complete the four directional functions

Implement move_left, move_right, move_up, and move_down. You may add helpers and adjust function signatures to report whether the board changed and how many points the move earned.

1. Slide nonzero tiles as far as possible in the chosen direction.
2. Merge equal adjacent tiles in the direction of travel. Each tile may participate in only one merge during a move. A newly merged tile cannot merge again in that move.
3. Close gaps after merging. Add the value of every newly merged tile to the score.
4. If the board changed, count one successful move and insert exactly one new 2 in a randomly selected empty cell. This includes a slide with no merge.
5. If the board did not change, do not insert a tile or change the score or move count.

Keep random insertion outside movement functions so movement can be tested deterministically.

Movement examples: before random insertion

Original row	Result of a left move	Score gained
0 2 0 2 0	4 0 0 0 0	4
2 2 2 0 0	4 2 0 0 0	4
2 2 2 2 2	4 4 2 0 0	8
2 2 4 0 0	4 4 0 0 0	4
4 4 8 8 0	8 16 0 0 0	24
2 4 8 0 0	2 4 8 0 0	0

A right move on 2 2 2 0 0 gives 0 0 0 2 4. Apply the same ordering rules to columns for up/down moves.

Required controls and display

Display commands for u/d/l/r (movement), s (save), o (open/load), n (new game after confirmation), h (help), and e (exit). Explain them in the program and README. Reject invalid commands and handle end of input cleanly.

Display the board, score, successful-move count, and highest tile at startup and after movement, a new game, or a successful load. Help, save, invalid input, and exit must not change the game state. Random insertion earns no score.

Announce success when a tile reaches 2048 or higher and allow continued play. Game over means there are no empty cells and no equal horizontal or vertical neighbors. A full board may still have a valid move. At game over, allow save, load, new game, help, and exit, but prevent further movement.

Part A: 2048 - save, load, and tests

Save and load requirements

Use C++ text-file streams and one slot named savegame.txt in the working directory. Saving again may replace this slot; explain that behavior in help and the README. Report file-open or write failures accurately.

Use this layout: the marker CS211_2048_V1; then the score and successful-move count; then five rows of five integers. For example:

```
CS211_2048_V1
8 3
4 4 0 0 0
0 0 0 0 0
2 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Save all 25 cells and both counters. Loading must:

- Check the version marker, two nonnegative integer counters, and exactly 25 integer cell values.
- Accept a cell only if it is zero or a power of two of at least 2. Reject missing data, extra non-whitespace content, and values outside the range of the chosen types.
- Read into temporary variables and a temporary board. Replace the active state only after the complete file passes validation.
- Leave the game unchanged if the file is missing or invalid, and display a useful error message.
- Restore the exact saved board and counters without inserting a tile or counting a move. The future random sequence does not need to be restored.

Required 2048 test evidence

Record starting state/input, expected result, actual result, and pass/fail in tests.md. Include board values. Separate the deterministic movement result from the later random insertion.

1. Sliding and merging in each of the four directions, including movement through gaps.
2. Three equal tiles and four equal tiles: no tile merges twice in one move.
3. A successful slide with no merge: one new 2, one counted move, no score gain.
4. An unchanged move: no inserted tile and no changed counters.
5. A full board with a possible merge, and a full board with no possible moves.
6. Several merges in one move, correct score updates, and reaching the 2048 target.
7. Save, change the game, and load: exact restoration of all cells and counters.
8. Missing, incomplete, and malformed saves, including an invalid tile: active state is preserved.
9. Help, invalid commands, new-game confirmation, and exit behavior.

Manual dry run

Include one 2048 movement dry run in report.pdf. Show the starting board, intermediate slide/merge steps, final board before random insertion, score gained, and whether a new tile should appear. Explain why the merge-once rule is satisfied.

Part B: Luhn checksum validation

Objective and input

Write a separate console program that checks a 16-digit number using the Luhn algorithm. Treat this as checksum validation: a passing checksum does not establish that a card exists or is authorized for payment.

Read the input as a string or character array so leading zeros are preserved. Accept exactly 16 characters, each a digit from 0 to 9. Reject spaces inside the number, letters, punctuation, and incorrect length with a clear input-format message. Distinguish malformed input from a correctly formatted number whose checksum fails.

Required algorithm

1. Starting at the rightmost digit, leave that digit unchanged and double every second digit as you move left.
2. If a doubled value has two digits, add those digits together. Subtracting 9 from a doubled value greater than 9 is equivalent.
3. Sum all the resulting values, including unchanged digits.
4. The number passes the Luhn checksum exactly when the sum is divisible by 10.

For example, the synthetic test input 4532015112830366 passes: the transformed values sum to 50. Changing its last digit to 7 produces a sum of 51 and fails.

Functions and interaction

Separate input-format validation from the checksum calculation. Include a clearly named checkLuhn function and any helpers needed. Implement the calculation yourself rather than calling a checksum library or external service.

Allow the user to check more than one number without restarting the program. Offer a clear exit choice. Reject malformed inputs and allow another attempt; handle end of input cleanly. Do not modify the original input while preparing the checksum calculation.

Required Luhn tests

Test input or action	Expected result
4532015112830366	Correct format; checksum passes
4532015112830367	Correct format; checksum fails
0000000000000018	Correct format; leading zeros preserved; passes
453201511283036	Reject: 15 digits
45320151128303666	Reject: 17 digits
453201511283036X	Reject: non-digit
Input containing an internal space	Reject: malformed input
Several checks followed by exit	Results correct; loop exits cleanly

Record expected and actual outcomes in tests.md.

Explanation and diagrams

In report.pdf, dry-run 4532015112830366 using a table with the original digit, whether it is doubled, its transformed value, and the running sum. State the traversal order and explain the final modulo check.

Include a flowchart for the Luhn program and a function workflow diagram showing main, input validation, checkLuhn, and helpers. Neat hand-drawn diagrams are acceptable.

Final package and submission

Folder structure

Submit one ZIP containing a top-level folder named CS211_Project_<session>_<group>. Replace the placeholders with your session and group identifier. A recommended structure is:

```
CS211_Project_MW_Group01/
    README.md
    report.pdf
    tests.md
    contributions.md
    group_info.txt  (Blackboard copy only)
    game2048/
        main.cpp
        2048_helper.h
        2048_helper.cpp
        sample_save.txt
    luhn/
        main.cpp
        luhn.h
        luhn.cpp
```

You may use other meaningful filenames and add helpers. Each program must have its own main function and build separately. Use include guards in headers, declarations in headers, and helper implementations in .cpp files. Do not include .cpp files using #include.

Build and run instructions

For the structure above, examples from the top-level project folder are:

```
g++ -std=c++17 -Wall -Wextra -pedantic game2048/main.cpp game2048/2048_helper.cpp -o game2048_app
g++ -std=c++17 -Wall -Wextra -pedantic luhn/main.cpp luhn/luhn.cpp -o luhn_app
```

Document the actual commands for your files and operating system, including how to run each program. Explain that savegame.txt is read/written in the current working directory. No absolute paths to your computer should be required.

Shared documentation

- README.md: names/GitHub usernames, session/group, repository URL, program summaries, build/run commands, controls, save-file behavior, and known limitations. Keep WSUIDs in group_info.txt in the Blackboard ZIP only.
- report.pdf: concise design explanation of both programs, one 2048 dry run, the Luhn dry-run table, and Luhn flowchart/function workflow diagram. Explain important functions and design choices; do not duplicate every source file in the report.
- tests.md: the required tests for both parts with expected/actual results and pass/fail. Also record the final clean-extraction build test.
- contributions.md: member responsibilities, PR links, author/reviewer/merger roles, and checks performed. Identify outside sources/tools used and how adapted code was checked. All members remain responsible for understanding the submission.

Before submitting

Extract the ZIP into a new directory and follow your README to build and run both programs. Include all source/header files, documentation, and a valid sample 2048 save. Exclude compiled executables, object files, IDE caches, and unrelated files. Check that the submitted source matches the behavior and dry runs in the report.

Submit the single group ZIP through the project submission link on Blackboard by the posted deadline, three weeks after release. Both programs are required. The separate practice-problems lab and GTA check-offs remain separate assessments.

Git and GitHub: install and get ready

Everyone: begin here after reading both problem descriptions

Git tracks versions of files on your computer. GitHub hosts the shared repository online. A commit records a change; a branch separates a task from main; a pull request (PR) asks teammates to review and combine that branch with main. A merge combines the approved work.

Use your own GitHub account for every contribution. Label your group members A, B, and, if applicable, C in your plan. These labels describe rotating roles, not separate accounts to create. Use one shared repository for both programs.

Step 1: create your account

1. Open GitHub signup, create an account if needed, and verify your email. Sign in using your own account.
2. Record your teammates' GitHub usernames. You will invite only your group members as collaborators.
3. On GitHub, open your profile menu, Settings, then Emails. If you want to keep your email private, use the GitHub-provided no-reply address for Git commits. Copy the exact address shown for your account.

Step 2: install Git on your computer

- Windows: open the official Git for Windows installer page, run the installer for your computer, and keep the standard options unless your lab setup requires otherwise. Open Git Bash from the Start menu afterward.
- macOS: open Terminal and run git --version. If Git is missing, run xcode-select --install and complete Apple's Command Line Tools installation. Reopen Terminal and check again. See the official macOS installation guide.
- Linux: use your distribution's package manager. On Ubuntu/Debian, run sudo apt update and then sudo apt install git. Other distributions are listed in the official Linux installation guide.

In Git Bash or Terminal, enter:

```
git --version
```

You should see a version number. If the command is not found, finish installation and reopen the terminal before proceeding. Git is separate from your C++ compiler.

Step 3: set your Git identity once

Replace the example name and email below with your own. Keep the quotation marks. Use your verified GitHub email or the exact no-reply address copied above.

```
git config --global user.name "Your Name"
git config --global user.email "your-verified-or-noreply-email"
git config --global init.defaultBranch main
git config user.name
git config user.email
```

The last two commands should print the identity you entered. This labels local commits; it does not sign you into GitHub. On a shared lab account, use a repository-local identity after cloning instead of changing another person's global settings.

GitHub Desktop is an optional graphical tool. The next pages use the GitHub website as the reference for branches, PRs, reviews, and merges; the Desktop alternative is explained later. Setup reference: First-time Git configuration.

Create the group's shared repository

Member A: create it on the GitHub website

1. Sign in to GitHub. Open the + menu near the top-right and select New repository.
2. Use a name such as cs211-project-mw-group01. Put both 2048 and Luhn in this one repository.
3. Public is preferred so the instructor can view progress and PR history using the repository link. A private repository is allowed; follow the visibility instructions below.
4. Select Add README. You may select the C++ .gitignore template. Click Create repository. Keep the default branch named main for this tutorial.
5. Open Settings, then Collaborators under Access. Select Add people and invite B and C, if present, using their GitHub usernames. Do not invite the instructor as a collaborator.
6. B and C: sign in to your own accounts and accept the invitation. Open the same repository link. You will all work in branches of this repository rather than creating separate copies on GitHub.

Success check: every member can open the repository and create a branch. See Create a repository and Invite teammates.

Visibility and information to share

Send the repository URL to the instructor through Blackboard by the end of Week 1 and keep the same repository throughout the project. Keep updates visible as you work rather than uploading the finished project only at the end.

A private URL alone does not let the instructor read its contents. If you keep the repository private, provide dated source snapshots and screenshots of commits, PR discussions, reviews, and merges with your progress updates and whenever the instructor requests them. Do not add the instructor as a collaborator. Public repositories allow direct inspection of updates without invitations.

Keep WSUIDs in group_info.txt in the Blackboard submission only. The public README may list names and GitHub usernames. Do not upload group_info.txt or other files containing student IDs to GitHub. The same separation applies to the report if you put student IDs on its cover page.

Everyone: clone one working copy to your computer

On the repository website, select Code, then HTTPS, and copy the repository URL. In Git Bash/Terminal, first change to the folder where you keep course projects. Replace the example URL and folder name below with your group's actual values.

```
git clone https://github.com/OWNER/cs211-project-mw-group01.git
cd cs211-project-mw-group01
git status
```

Clone downloads the files and Git history. Run later project commands inside this folder. Do not run git init inside the clone. If using a shared lab login, set git config user.name and git config user.email without --global here.

For a private clone or later pushes, use the browser sign-in offered by Git Credential Manager or GitHub Desktop. If your terminal asks for an account password, your normal GitHub password will not authenticate Git over HTTPS; use the official HTTPS credential guide or Desktop's sign-in rather than guessing passwords.

Checkpoint 1: your first branch and PR

Member A: prepare a small setup change

1. On the repository's Code tab, check that the branch selector says main. Open that selector, type setup/project-plan, and choose Create branch. You are now viewing the new branch.
2. Open README.md and click the pencil icon. Add a short description of both programs, the members' GitHub usernames, and your task plan. Do not include WSUIDs.
3. Select Commit changes. Use a message such as Add group plan and project overview. Confirm that the destination is setup/project-plan, then commit.
4. On the same branch, choose Add file, then Upload files. Upload the starter files inside a folder named game2048. Preserve main.cpp and 2048_helper.h; do not upload the starter ZIP as the source code. Commit this change to setup/project-plan.
5. Use Add file, then Create new file to add luhn/README.md with your Luhn implementation plan. Typing the slash creates a folder. Commit to the same branch. main still contains the original README until you merge the PR.

Use an editor and your usual C++ environment for coding. You may edit text on the website or upload tested local files to the correct branch. The website stores and compares code; this tutorial does not require it to compile C++ for you.

Member A: open the pull request on the website

1. Open Pull requests, then New pull request. Set base to main and compare to setup/project-plan. Base is the destination; compare is the branch containing your changes.
2. Read the comparison and confirm that only the intended files changed. Click Create pull request. If GitHub shows Compare & pull request after a commit, that is another route to the same form.
3. Give the PR a specific title, such as Set up folders and group plan. Explain what changed, what you checked, and what remains unfinished. For this first PR, clearly say that the starter movement functions and Luhn program are still incomplete.
4. Create the PR as ready for review. Under Reviewers, select B. Send B the PR link. Do not merge your own work immediately; B must review it first.

Use this description pattern for later code PRs:

```
Completed: left sliding and merging
Checked: gaps, three equal tiles, merge-once behavior
Expected/actual: list the input and result for each test
Still to do: other directions and full game controls
Reviewer: teammate's GitHub username
```

Success check: your PR is open, its base is main, and the setup branch contains the changes. Move to the next page for B's review and the first merge. Help: Branches, edit files, and upload files.

Checkpoint 1: teammate review and merge

Member B: review A's work using your own account

1. Open A's PR link on GitHub. Read the description and select Files changed. Check the added lines, file locations, names, and whether the plan covers both programs.
2. Leave a specific comment or question, such as The plan assigns Luhn testing to me; who will test save/load? A useful review explains what you checked rather than only saying Looks good.
3. If a correction is needed, select Review changes, choose Request changes, describe the correction, and Submit review. A makes another commit on setup/project-plan. The existing PR updates automatically; A does not open a replacement PR for the fix.
4. After checking the correction, select Review changes, choose Approve, add a brief summary, and Submit review. A PR author cannot approve their own PR.

For code PRs, B must also compile and run the proposed branch before approving. To get it without Git commands, open Code, select the PR's branch, then Code > Download ZIP. Extract it to a separate test folder and build the relevant program. Record which inputs passed or failed in the review. Do not test main and assume it includes an unmerged PR.

Another member: merge after approval

For a two-person group, B reviews and merges PR1. For a three-person group, C merges after B approves. Open the PR's Conversation tab and verify that the review is submitted and questions are resolved.

1. At the merge controls, use the dropdown to select Create a merge commit. Click Merge pull request, then Confirm merge. Use this method for the six project checkpoints so their merges remain visible in history.
2. Check for the purple Merged status. Closing a PR without merging is not the same action. You may delete the finished feature branch after the merge; keep main.
3. Open the repository's Code tab and select main. Confirm that A's new files and README now appear there. Each member should open main and check the result.
4. In your local clone, finish or save outstanding work appropriately, then update using the commands below. Run them only with a clean working tree; git status should report nothing to commit. If it lists work you still need, stop and preserve it before switching branches.

```
git status
git switch main
git pull --ff-only origin main
```

main on your computer should now match the merged work on GitHub. Create each new task branch from the latest main, not from an old feature branch. A merge on the website does not automatically update local files.

Success check: PR1 says Merged, a teammate submitted a review, main contains the setup work, and everyone's working copy has been updated. See Review a PR and merge a PR.

Follow these checkpoints while coding

Complete at least six meaningful PRs and merge them into main during the project. Use the full branch > commit > PR > teammate review > merge > update cycle each time. Every member must author a substantive code PR, review a teammate's work, and merge a teammate's approved PR. Use your own account for each role.

Checkpoint 2: once move_left works

Create feature/move-left from main. Complete left sliding and merging and add the row-example tests from Part A. Keep the random insertion separate. Open the PR after those tests pass. The reviewer must try three equal tiles and the 2 2 4 0 0 example. A teammate approves and merges; everyone updates main before the next task.

Checkpoint 3: once all directions and controls work

Create feature/other-directions from the updated main. Complete right/up/down, tile insertion, scoring, the menu, and win/game-over behavior. Include vertical tests and an unchanged move. Open a PR with the actual outputs. Another member tests the branch and reviews it before a teammate merges. Split this into more PRs if useful; six is a minimum, not a reason to combine unrelated changes.

Checkpoint 4: once save/load works

Create feature/save-load from main. Add persistence and run the valid, missing, and malformed save tests. Open a PR with evidence that a failed load leaves the active game unchanged. The reviewer repeats that check and the save/change/load round trip. Approve, merge, and update everyone's main.

Checkpoint 5: once Luhn works

Read Part B and finish Luhn input validation, checksum calculation, and repeat-input interaction. Before starting, create feature/luhn from the latest main. Commit progress as you work. Once the required Luhn tests pass, open a PR to main with test results. Another member downloads or checks out that branch, runs the tests, and reviews it. A teammate merges after approval. Everyone updates main.

Checkpoint 6: once the final package is ready

Create docs/final-package from main. Finish the public README, test record, diagrams/report without student IDs, and contribution notes. Ask another member to build both programs from this branch in a clean test folder. Fix any problems on the same branch, then get a fresh review of the changes and merge. Everyone updates main again before preparing the Blackboard ZIP.

Rotate the roles

For two members: A authors checkpoints 1, 3, and 5; B authors 2, 4, and 6. The other member reviews and merges each PR. For three members: A authors 1 and 4, B authors 2 and 5, C authors 3 and 6. The next member reviews and the third merges: A > B > C > A.

Keep the PR URL, author, reviewer, merger, and tested examples in contributions.md. By the final submission, include evidence that all six PRs were merged. Work should appear throughout the three weeks. Do not wait until the final day to manufacture six copies of a finished upload.

Working locally without losing changes

Choose one editing route for a task

Website route: select your task branch on GitHub; edit with the pencil icon or upload changed source files into their existing folders. Commit to that task branch. To test, download its ZIP or update the local tracking branch. After local fixes, upload only the files you intentionally changed to the same branch. Read the diff before committing so an older file does not overwrite a teammate's newer work.

Git route: create the task branch on the website first. In your clean local clone, replace feature/luhn below with the branch you created:

```
git fetch origin
git switch --track origin/feature/luhn
```

Use --track only the first time you bring that branch onto this computer. On later visits use git switch feature/luhn, then git pull --ff-only origin feature/luhn. Check git status before changing branches. Edit and compile inside your clone, then commit only the intended files. For example:

```
git status
git diff
git add luhn/main.cpp luhn/luhn.h luhn/luhn.cpp
git commit -m "Implement Luhn validation and input loop"
git push origin feature/luhn
```

git add selects files for the next snapshot; commit records it locally; push sends it to GitHub. Now open and review the PR on the website as before. Do not also upload another copy of the same local change through the browser. If you edited that branch online instead, pull its new commits before editing it locally.

If GitHub reports a merge conflict

A conflict can occur when two branches edit the same lines. It does not mean that the whole project is lost. First agree with the other author on what the final file should contain.

1. Open the affected PR on the website. If Resolve conflicts is available, open it. Keep the intended final content and remove the conflict-marker lines beginning <<<<<<<, =======, and >>>>>>>.
2. Choose Mark as resolved for each affected file, then Commit merge. This updates the PR's feature branch with the resolution; it does not finish merging the PR into main.
3. Download or update the resolved feature branch and compile/test again. Have the reviewer check the resolution, approve it, then merge the PR into main normally.
4. If GitHub cannot resolve the conflict in the browser, ask the GTA for help or preserve your work and resolve it with Git/Desktop. Do not delete the repository or discard files to make the warning disappear.

Avoid conflicts by merging small tasks regularly, starting new branches from current main, and agreeing who edits shared files. If a pull fails because local and remote work diverged, stop and ask for help; do not use a force push or destructive reset as a shortcut.

Track the work on the website

The Code tab shows files for the selected branch. Open a file's History to see its commits and compare changes. Pull requests shows the discussion, Files changed, reviews, and merge result; use the Closed list or search is:pr is:merged to find completed PRs. A ZIP download contains files, not the full Git history or PR discussion. Conflict reference: Resolve a conflict on GitHub.

Optional: GitHub Desktop and final checks

Use Desktop for local Git operations if you prefer

The required workflow and roles stay the same. The website steps above remain the reference for PR creation, review, and merging. Desktop provides another way to bring files to your computer and record changes.

1. Install GitHub Desktop on Windows or macOS and sign in to your own GitHub account. Check the name and email used for commits. This tutorial's command-line Git setup remains useful even if you choose Desktop.
2. Choose File > Clone repository, select your group's existing repository or enter its URL, choose a local folder, and clone it. Do not create a second repository with the same project name. If using an existing command-line clone, add that local repository to Desktop instead of cloning over it.
3. Before a new task, select main under Current Branch. Click Fetch origin, then Pull origin if new commits are available. With a clean working copy, choose Current Branch > New Branch and name it for the checkpoint. Click Publish branch to share it.
4. Edit and test in your C++ environment. In Desktop's Changes tab, inspect the diff, select the intended files, enter a meaningful summary, and click Commit to the named task branch. Click Push origin.
5. Open the repository on the GitHub website. Create the PR from that task branch into main, request the assigned teammate's review, and follow the website review/merge steps. Desktop's Preview Pull Request/Create Pull Request can also open the browser for this step.
6. After a teammate merges, return to Desktop, switch to main, Fetch origin, then Pull origin. Confirm the merged changes before beginning another task.

Desktop references: Clone and get started, commit changes, and sync branches.

Before each PR

- The branch is named for the task and the PR's base is main.
- Only intended source/documentation files changed. Compiled executables, object files, student IDs, and unrelated files are excluded. Browser uploads require you to select files carefully; a .gitignore does not filter what you upload manually.
- The description says what changed, gives concrete expected/actual test results, and identifies unfinished work.
- Another group member reads and tests the proposed change before approving. The reviewer explains at least one check they performed.

Before the final ZIP

- At least six meaningful PRs show Merged, with author/reviewer/merger roles rotated across members.
- Both programs are complete on main, not only on an unmerged branch. All members can explain both programs.
- The README includes the repository URL. contributions.md lists the PR URLs and roles; include final merged-PR screenshots in the Blackboard submission for a private repository.
- The Blackboard ZIP includes group_info.txt with names/WSUIDs, but that file is excluded from GitHub. Download the final main or update your clone, add this private information only to a separate submission copy, and then test and ZIP that copy.
- The final build instructions work from a newly extracted folder. The repository URL supplements the required Blackboard ZIP; it does not replace it.

Use the earlier specifications as you work: Part A movement at checkpoints 2-3, Part A save/load at checkpoint 4, Part B Luhn at checkpoint 5, and final packaging at checkpoint 6. Repeat the review/merge/update cycle at each checkpoint.